

Interview Preparation Guide: Mohit's Kitchen (AI Recipe Generator – FastAPI Edition)

1. PROJECT OVERVIEW

What is Mohit's Kitchen?

Mohit's Kitchen is a modern, responsive full-stack web application that leverages Generative Artificial Intelligence (Groq Cloud API with the Qwen **qwen/qwen3.8-27b** model) to generate custom, step-by-step culinary recipes based on either a **dish name** or a list of **available ingredients**.

What Problem Does It Solve?

- **Food Waste Reduction:** Users often have random ingredients in their pantry or refrigerator and don't know what to make. Mohit's Kitchen turns available items into complete recipes.
- **Instant Meal Planning:** Eliminates the effort of manually searching through dozens of ad-heavy blog posts to find a simple, clear recipe.
- **Structured Culinary Instructions:** Generates standardized, easy-to-follow output including ingredients list, step-by-step preparation instructions, estimated cooking time, and difficulty level.

How Does the Application Work?

1. The user opens the React frontend UI and selects one of two input modes: **Dish Name** (e.g., "Pasta Carbonara") or **Ingredients** (e.g., "Eggs, Cheese, Pasta, Guanciale").
2. The user submits the form. React sends an asynchronous **POST** request (**/api/recipe/dish** or **/api/recipe/ingredients**) containing JSON payload to the FastAPI backend.
3. FastAPI intercepts the request, runs strict validation through **Pydantic** models, and passes the validated data to the **Groq Service layer**.
4. The Groq Service formats system and user prompts and calls the **Groq Cloud API**, requesting a strict JSON object from the **qwen/qwen3.8-27b** model.
5. Groq returns the generated JSON object. FastAPI validates the structured response and sends it back to the React frontend.
6. React parses the JSON and renders an animated, visually rich **Recipe Card** complete with difficulty badges, cooking time, ingredients checklist, and numbered instructions.

Main Features

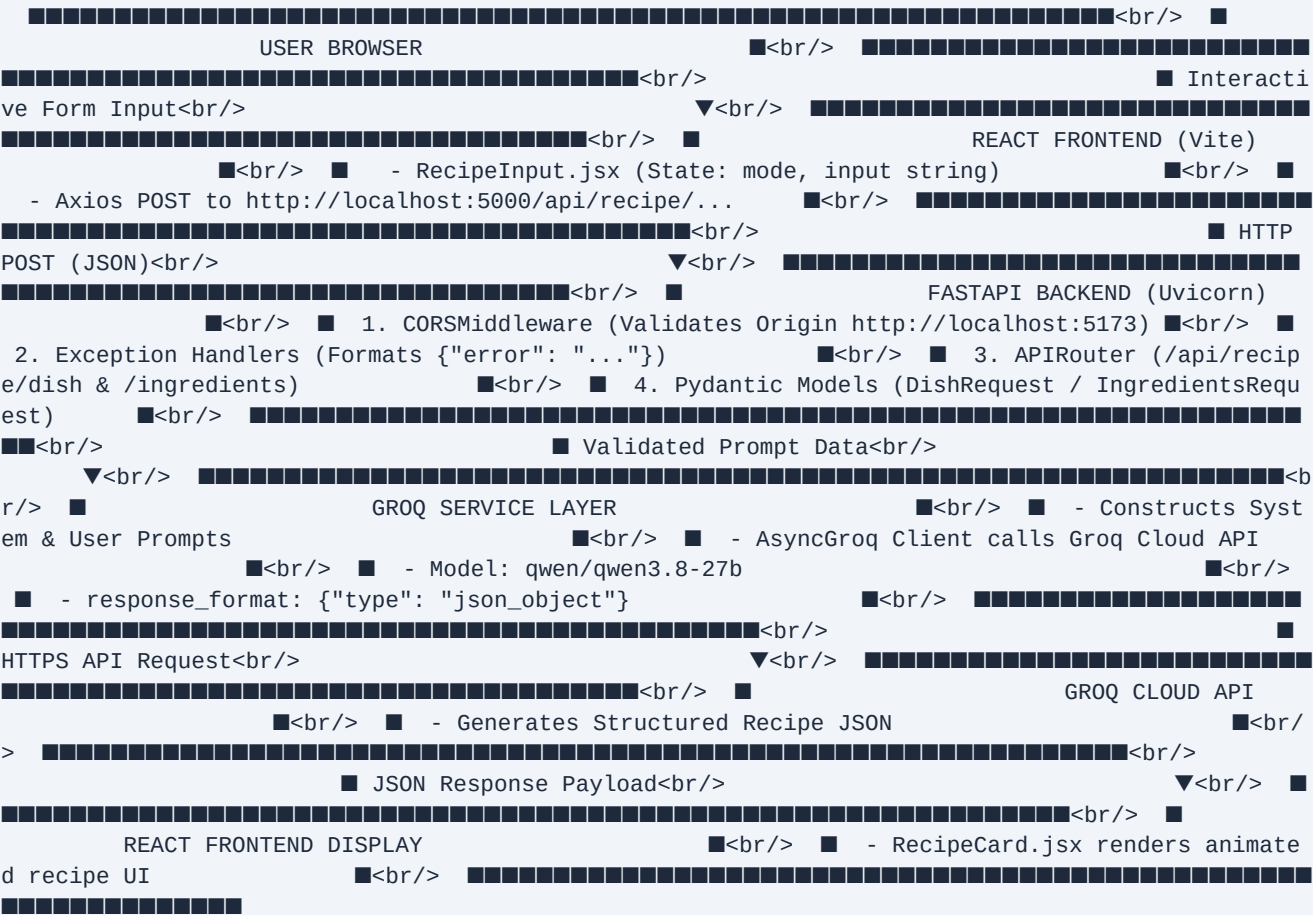
- **Dual Input Modes:** Generate recipes by dish name or pantry ingredients.
- **Strict Structured Output:** Guarantees standard fields (**recipeName**, **ingredients**, **instructions**, **estimatedCookingTime**, **difficultyLevel**).
- **Interactive UI:** Smooth animations powered by Framer Motion, Lucide icons, and responsive stateful UI.
- **Fast & Validated Backend:** Asynchronous Python FastAPI backend with Pydantic request validation.
- **Auto-Generated API Documentation:** Interactive Swagger UI accessible at **/docs**.

- **Health Check & CORS Support:** `/health` endpoint for monitoring and CORS middleware for frontend communication.
- **Environment Secret Management:** API key loaded securely via `.env` without exposure to frontend or source code.

Complete Technology Stack

Layer	Technology	Purpose
Frontend UI	React 18	Declarative component-based UI rendering
Build Tool	Vite	Lightning-fast frontend bundling and HMR dev server
Animations	Framer Motion	Smooth page transitions, skeleton loaders, card entry
Iconography	Lucide React	Modern visual UI icons
Styling	Vanilla CSS	Responsive layouts, CSS variables, glassmorphism
HTTP Client	Axios	Frontend-to-Backend asynchronous HTTP requests
Backend Framework	Python 3 + FastAPI	High-performance asynchronous REST API backend
Data Validation	Pydantic v2	Strict request parsing and response modeling
ASGI Web Server	Uvicorn	Asynchronous server implementation for running FastAPI
Environment	python-dotenv	Loading hidden configuration secrets from <code>.env</code>
AI Cloud API	Groq Python SDK (AsyncGroq)	Ultra-fast LLM inference API
LLM Model	Qwen 27B (qwen/qwen3.8-27b)	High-reasoning chat model for structured JSON recipes

High-Level Architecture Diagram



2. COMPLETE REQUEST FLOW

Below is the step-by-step breakdown of what happens when a user types "Pasta Carbonara" and clicks "Generate Recipe":

- 1. User Action:** The user types "Pasta Carbonara" into the search box and clicks the submit button.
- 2. React State Update:** `RecipeInput.jsx` captures the input and invokes `onGenerate('dish', 'Pasta Carbonara')` defined in `App.jsx`. `App.jsx` sets `loading = true`, clearing previous errors or recipes, triggering the animated `SkeletonLoader`.
- 3. HTTP Dispatch:** `Axios` fires an asynchronous **POST** request to `http://localhost:5000/api/recipe/dish` with payload: `{"dishName": "Pasta Carbonara"}`.
- 4. FastAPI CORS Check:** FastAPI's `CORSMiddleware` inspects the request's `Origin` header (`http://localhost:5173`). Matching the allowed list, it permits the request.
- 5. Pydantic Validation:** The endpoint signature `get_recipe_by_dish(payload: DishRequest)` automatically parses the incoming JSON. `Pydantic` verifies that `dishName` is present and is a non-empty string. If missing, a **400 Bad Request** with `{"error": "Invalid request body"}` is automatically returned.
- 6. Router Execution:** `routes/recipe.py` calls `generate_recipe_from_dish("Pasta Carbonara")` inside `services/groq_service.py`.

7. **Groq Service Assembly:** `groq_service.py` fetches **GROQ_API_KEY** from environment variables, instantiates **AsyncGroq()**, and constructs the system prompt demanding strict JSON return format alongside the user prompt **"Generate a complete recipe for the dish: Pasta Carbonara."**.

8. **Groq Cloud API Call:** **AsyncGroq** sends an HTTP POST request to **https://api.groq.com/openai/v1/chat/completions** specifying:

* **model:** "qwen/qwen3.8-27b"

* **response_format:** {"type": "json_object"}

* **temperature:** 0.7

9. **LLM Inference:** The Qwen model generates a JSON string containing **recipeName**, **ingredients**, **instructions**, **estimatedCookingTime**, and **difficultyLevel**.

10. **Backend JSON Parsing:** `groq_service.py` executes **json.loads()** on the raw LLM string to convert it into a Python dictionary.

11. **FastAPI Response Serialization:** FastAPI validates the dictionary against **RecipeResponse** Pydantic model and returns an HTTP **200 OK** JSON response to React.

12. **React Rendering:** Axios receives **response.data**. **App.jsx** updates **recipe = response.data** and sets **loading = false**. **AnimatePresence** smoothly replaces the skeleton loader with the rendered **RecipeCard** component.

3. FRONTEND DEEP DIVE

Core Technologies

- **React 18:** A JavaScript library for building user interfaces using declarative components and reactive state.
- **Vite:** A modern frontend build tool providing an ultra-fast HMR (Hot Module Replacement) development server and optimized Rollup production builds.
- **Framer Motion:** An animation library for React used for micro-interactions, modal entries, and smooth skeleton transitions.
- **Lucide React:** Icon library providing vector icons like **ChefHat**, **Clock**, **Flame**, etc.

Component Structure

```
frontend/src/<br>■■■■ main.jsx          # Mounts React root onto index.html DOM element<br>■■ App.jsx          # Master container holding main state (recipe, loading, error)<br>x.css              # Custom styling, color palette, glassmorphism design tokens<br>■■■■ components/<br>■■■■ RecipeInput.jsx # Toggle mode (Dish / Ingredients) and form input<br>■■■■ RecipeCard.jsx # Card displaying structured recipe result<br>■■■■ Footer.jsx     # Footer component with branding links
```

Key React Concepts Used

1. Components & Props

- **What:** Modular, self-contained UI building blocks that accept input data called "props".
- **How in Project:** **App.jsx** passes **onGenerate** callback prop to **RecipeInput.jsx**, and passes the **recipe** object prop to **RecipeCard.jsx**.

2. useState Hook

- **What:** A React Hook that enables components to maintain and update internal state, triggering re-renders upon change.
- **How in Project:** `App.jsx` uses 3 key states:

```
* const [recipe, setRecipe] = useState(null);
```

```
* const [loading, setLoading] = useState(false);
```

```
* const [error, setError] = useState('');
```

3. Event Handling & Asynchronous API Requests

- **What:** Capturing user form submit actions and triggering asynchronous HTTP requests.
- **How in Project:** `handleGenerateRecipe` in `App.jsx` executes an async `axios.post()` call wrapped in `try...catch...finally` blocks to safely handle loading indicators and display errors.

Interview Questions & Short Answers (Frontend)

Q: Why use React instead of plain HTML/JavaScript?

- **Answer:** React allows us to create reusable UI components and handle dynamic state updates seamlessly. When the backend returns a recipe, React automatically updates the DOM without needing full page reloads.

Q: Why use Vite over Create React App (CRA)?

- **Answer:** Vite uses native ES modules during development, offering near-instant server start and extremely fast Hot Module Replacement (HMR) compared to CRA's slow Webpack bundler.

Q: How is the API URL configured in the frontend?

- **Answer:** It uses Vite's environment variable `import.meta.env.VITE_API_BASE_URL` with a fallback to `http://localhost:5000`, ensuring flexible deployment across local and production environments.

4. FASTAPI BACKEND DEEP DIVE

What is FastAPI?

FastAPI is a high-performance, modern Python web framework designed for building RESTful APIs based on standard Python type hints. It is built on top of **Starlette** (for web routing) and **Pydantic** (for data validation).

Why FastAPI? (Comparison Table)

Feature	FastAPI	Flask	Express.js (Node.js)
Language	Python 3	Python 3	JavaScript (Node.js)
Asynchronous (Async/Await)	Native (built on ASGI)	WSGI (Sync by default)	Native (Event loop)
Data Validation	Automatic via Pydantic	Manual / Third-party	Manual / Third-party (Joi)
Auto API Docs (Swagger)	Built-in (/docs)	None (Manual setup)	None (Manual setup)
Type Safety	High (Python type hints)	Low	Low (unless TypeScript used)
Performance	Extremely High	Moderate	High

Core REST API Concepts

- **API (Application Programming Interface):** A software intermediary allowing two applications (React and Python) to talk to each other.
- **REST (Representational State Transfer):** An architectural style for designing networked applications using standard HTTP methods.
- **HTTP Methods Used:**

* **GET:** Used for retrieving data without mutating state (**GET /**, **GET /health**, **GET /docs**).

* **POST:** Used for submitting data to be processed (**POST /api/recipe/dish**, **POST /api/recipe/ingredients**).

APIRouter & Modular Routing

FastAPI provides **APIRouter** to break down endpoints into separate domain modules. In our project:

```
# routes/recipe.py<br/>from fastapi import APIRouter<br/><br/>router = APIRouter(prefix="/api/recipe", tags=["Recipe"])
```

This keeps **main.py** lightweight:

```
# main.py<br/>app.include_router(recipe_router)
```

Uvicorn & ASGI

- **ASGI (Asynchronous Server Gateway Interface):** The modern Python standard for async web servers.
- **Uvicorn:** The lightning-fast ASGI web server that runs FastAPI applications:

```
`bash
```

```
uvicorn main:app --reload --port 5000
```

```
`
```

Detailed Walkthrough of Endpoint Execution: POST /api/recipe/dish

```
@router.post("/dish", response_model=RecipeResponse)<br/>async def get_recipe_by_dish(payload: DishRequest):<br/>    if not payload.dishName or not payload.dishName.strip():<br/>        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Dish name is required")<br/>    try:<br/>        recipe = await generate_recipe_from_dish(payload.dishName)<br/>        return recipe<br/>    except ValueError as ve:<br/>        raise HTTPException(status_code=400, detail=str(ve))<br/>    except Exception as e:<br/>        raise HTTPException(status_code=500, detail=f"Error generating recipe: {str(e)}")
```

- 1. Routing:** POST /api/recipe/dish matches the @router.post("/dish") handler.
- 2. Body Extraction & Validation:** FastAPI deserializes the request body into **payload: DishRequest**. If **dishName** is missing, Pydantic immediately rejects it with a **400 Bad Request**.
- 3. Empty String Guard:** The function checks if **dishName.strip()** is empty and raises an **HTTPException(400)**.
- 4. Service Call:** **await generate_recipe_from_dish(...)** triggers the Groq Service asynchronously without blocking the event loop.
- 5. Response Serializing:** The returned dictionary is validated against **RecipeResponse** and converted into JSON.

5. PYDANTIC DEEP DIVE

What is Pydantic?

Pydantic is the data validation and settings management library used by FastAPI. It enforces type hints at runtime and provides user-friendly errors when data is invalid.

Schemas Implemented in Project (schemas/recipe.py)

```
from typing import List, Optional<br/>from pydantic import BaseModel, Field<br/><br/>class DishRequest(BaseModel):<br/>    dishName: str = Field(..., description="Name of the dish to generate a recipe for")<br/><br/>class IngredientsRequest(BaseModel):<br/>    ingredients: str = Field(..., description="Available ingredients string")<br/><br/>class RecipeResponse(BaseModel):<br/>    recipeName: str<br/>    ingredients: List[str]<br/>    instructions: List[str]<br/>    estimateCookingTime: str<br/>    difficultyLevel: str<br/><br/>class ErrorResponse(BaseModel):<br/>    error: str<br/>    details: Optional[str] = None
```

Why Pydantic is Better than Manual Checks

- **Automatic Type Casting & Validation:** Automatically checks if strings are strings, lists are lists, etc.
- **Standardized Errors:** Generates clean validation failure outputs.
- **Auto Documentation:** Automatically populates request/response schemas in Swagger UI (/docs).

Interview Questions & Short Answers (Pydantic)

Q: What happens if a user sends {"dishName": 12345}?

- **Answer:** Pydantic will attempt to safely coerce **12345** to string **"12345"**. If it cannot satisfy the schema, FastAPI automatically catches the **RequestValidationError** and returns an HTTP 400 Bad Request.

Q: How does FastAPI use Pydantic for response validation?

- **Answer:** By setting `response_model=RecipeResponse` on the route decorator, FastAPI filters the outgoing dictionary, ensuring only declared fields are returned in the exact expected types.

6. PROJECT STRUCTURE / SEPARATION OF CONCERNS

```

backend/
├── main.py                # Application entry point, CORS, exception handlers, endpoints
├── config.py              # Environment configuration (python-dotenv)
├── requirements.txt       # Dependency manifest
├── .env                   # Private secrets (GROQ_API_KEY)
├── .env.example           # Environment template
├── routes/                # Route handlers & HTTP validation logic
│   ├── recipe.py          # AI integration & LLM prompt logic
│   └── groq_service.py    # AI integration & LLM prompt logic
├── schemas/               # Pydantic request/response data models
└── recipe.py              # Pydantic request/response data models

```

Why Separate into Layers? (Separation of Concerns)

- **Maintainability:** Modifying Groq AI prompts only requires editing `groq_service.py` without touching route code or Pydantic models.
- **Testability:** Each layer can be tested independently.
- **Scalability:** Adding new endpoints (e.g., nutrition analyzer) requires simply adding a router without bloating `main.py`.

7. GROQ + LLM INTEGRATION

What is Groq?

Groq is an AI infrastructure company that builds custom hardware called **LPUs (Language Processing Units)**. LPUs execute Large Language Models at ultra-high inference speeds (hundreds of tokens per second).

What is the Qwen Model?

In this project, we use the **qwen/qwen3.8-27b** model provided via Groq Cloud API. It is a highly capable chat model optimized for instruction following and structured JSON outputs.

Implementation in services/groq_service.py

```
import json<br/>import os<br/>from groq import AsyncGroq<br/>from dotenv import load_dotenv<br/>
<br/>load_dotenv()<br/><br/>SYSTEM_PROMPT = """"You are an expert chef. Return a JSON object with
the following structure:<br/>{<br/>  "recipeName": "String",<br/>  "ingredients": ["String"],<br/>
r/>  "instructions": ["String"],<br/>  "estimatedCookingTime": "String (e.g. 30 mins)",<br/>  "d
ifficultyLevel": "String (Easy, Medium, Hard)"<br/>}<br/>Return STRICTLY valid JSON ONLY. Do not
use markdown text around the json, just the raw JSON string.""<br/><br/>def get_groq_client()
-> AsyncGroq:<br/>    api_key = os.getenv("GROQ_API_KEY", "")<br/>    if not api_key or api_key.
strip() == "your_api_key_here":<br/>        raise ValueError("GROQ_API_KEY is missing or invalid
in environment variables.")<br/>    return AsyncGroq(api_key=api_key)<br/><br/>async def genera
te_recipe(user_prompt: str) -> dict:<br/>    client = get_groq_client()<br/>    try:<br/>
    response = await client.chat.completions.create(<br/>        model="qwen/qwen3.8-27b",<br/>
        messages=[<br/>            {"role": "system", "content": SYSTEM_PROMPT},<br/>
            {"role": "user", "content": user_prompt}<br/>        ],<br/>        respons
e_format={"type": "json_object"},<br/>        temperature=0.7,<br/>        )<br/>    con
tent = response.choices[0].message.content<br/>    return json.loads(content)<br/>    except
json.JSONDecodeError:<br/>        raise ValueError("Failed to parse valid JSON from Groq AI res
ponse")<br/>    except Exception as e:<br/>        raise RuntimeError(f"Failed to generate recip
e from Groq AI: {str(e)}")
```

8. PROMPT ENGINEERING

System Prompt vs. User Prompt

- **System Prompt:** Instructs the LLM on its **role**, output **format constraints**, and **schema requirements**.
- **User Prompt:** Provides the dynamic runtime request (e.g., "**Generate a complete recipe for the dish: Pasta Carbonara.**").

Why response_format={"type": "json_object"} is Crucial

Without JSON mode, LLMs often wrap outputs in Markdown blocks (e.g., json ...) or append conversational preamble ("**Here is your recipe:**"). Enabling `response_format={"type": "json_object"}` forces the model to produce raw, parsable JSON string output compatible with `json.loads()`.

9. ERROR HANDLING

Comprehensive Error Matrix

Error Scenario	Internal Root Cause	HTTP Status Code	Response Body Format
Empty Input	<code>dishName</code> or <code>ingredients</code> is empty/blank	400 Bad Request	<code>{"error": "Dish name is required"}</code>
Invalid Payload	Missing required JSON keys	400 Bad Request	<code>{"error": "Invalid request body", "details": "..."} </code>
Missing API Key	<code>GROQ_API_KEY</code> missing in <code>.env</code>	400 Bad Request	<code>{"error": "GROQ_API_KEY is missing or invalid..."}</code>
Groq API Error	Network timeout or 401 Auth Failure	500 Internal Error	<code>{"error": "Error generating recipe: ..."} </code>
Invalid AI JSON	LLM returned malformed JSON	400 Bad Request	<code>{"error": "Failed to parse valid JSON..."}</code>

Custom Exception Handlers (main.py)

To maintain compatibility with the React frontend expecting `{"error": "..."}` :

```
@app.exception_handler(HTTPException)
async def http_exception_handler(request: Request, exc: HTTPException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": exc.detail})
```

10. CORS (CROSS-ORIGIN RESOURCE SHARING)

What is CORS?

CORS is a browser security mechanism that restricts web applications running at one origin (`http://localhost:5173`) from making HTTP requests to a server on a different origin (`http://localhost:5000`).

Implementation in main.py

```
from fastapi.middleware.cors import CORSMiddleware

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
    "http://localhost:3000",
    "http://127.0.0.1:3000",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

11. ENVIRONMENT VARIABLES & SECURITY

- **.env File:** Stores sensitive keys (`GROQ_API_KEY=gsk_...`) locally. Added to `.gitignore` so secrets are never pushed to GitHub.
 - **.env.example File:** Safe template committed to repository so developers know which keys to configure.
 - **Loading Mechanism:** `python-dotenv` loads `.env` variables into `os.environ` on application startup.
-

12. API DOCUMENTATION / SWAGGER

FastAPI automatically reads Pydantic schemas, route definitions, and docstrings to generate interactive documentation accessible at `/docs` (Swagger UI) and `/redoc` (ReDoc).

Testing Endpoints via Swagger

1. Open `http://localhost:5000/docs` in browser.
 2. Select **POST** `/api/recipe/dish`.
 3. Click "Try it out", enter `{"dishName": "Pasta Carbonara"}`, and click **Execute**.
 4. View live status code (**200**), response headers, and output JSON body.
-

13. COMMON TECHNICAL INTERVIEW QUESTIONS & ANSWERS

A. Project Basics

Q1: Tell me about your project "Mohit's Kitchen".

- **Answer:** Mohit's Kitchen is an AI-powered full-stack recipe generator built with React and Python FastAPI. It communicates with Groq Cloud API using the Qwen `qwen/qwen3.8-27b` model to generate structured culinary recipes based on dish names or available pantry ingredients.

Q2: What problem does your application solve?

- **Answer:** It solves pantry indecision and food waste by letting users generate complete recipes from ingredients they already have, avoiding ad-heavy blog searches.

B. React

Q3: How do you pass data between components in React?

- **Answer:** Via props. For instance, `App.jsx` passes state and handler callbacks down to `RecipeInput.jsx` and the generated recipe object to `RecipeCard.jsx`.

Q4: How do you handle loading states in the frontend?

- **Answer:** `App.jsx` toggles a boolean `loading` state during Axios API calls. When `true`, an animated Framer Motion `SkeletonLoader` is displayed.

C. FastAPI

Q5: Why did you choose FastAPI over Express or Flask?

- **Answer:** FastAPI offers high performance through native Python async support, automatic request validation using Pydantic, and automatic Swagger documentation generation out-of-the-box.

Q6: What is an ASGI server?

- **Answer:** ASGI (Asynchronous Server Gateway Interface) is the standard interface for async Python web applications. We use Uvicorn as our ASGI server.

D. REST APIs

Q7: What is the difference between GET and POST methods?

- **Answer:** **GET** is used to retrieve data without modifying server state, whereas **POST** sends data in the request body to be processed.

Q8: What HTTP status codes does your API return?

- **Answer:** **200 OK** for success, **400 Bad Request** for missing/invalid input, and **500 Internal Server Error** for upstream API failures.

E. Pydantic

Q9: What is Pydantic and how is it used in FastAPI?

- **Answer:** Pydantic is a data validation library that enforces type hints at runtime. FastAPI uses it to automatically parse, validate, and serialize HTTP request/response payloads.

Q10: What happens if required fields are missing in a request payload?

- **Answer:** Pydantic catches the missing field and FastAPI returns an HTTP 400 Bad Request with a clear error payload.

F. Groq & LLM

Q11: What is Groq?

- **Answer:** Groq is an AI compute platform utilizing custom LPUs (Language Processing Units) to achieve extremely fast LLM inference speeds.

Q12: Which LLM model does your backend use?

- **Answer:** We use **qwen/qwen3.8-27b**, a 27-billion parameter chat model accessible via Groq Cloud API.

G. Prompt Engineering

Q13: What is the difference between system prompt and user prompt?

- **Answer:** System prompt sets the persona, rules, and output format constraints. User prompt delivers the specific user input data.

Q14: Why do you enforce JSON mode in the Groq request?

- **Answer:** Setting `response_format={"type": "json_object"}` guarantees raw parsable JSON output without markdown backticks or conversational text.

H. CORS**Q15: What is CORS and why is it needed?**

- **Answer:** CORS is a browser security mechanism blocking requests across different origins. Since React runs on port 5173 and FastAPI on port 5000, **CORSMiddleware** explicitly allows requests from the frontend origin.

I. Security**Q16: How do you secure the Groq API key?**

- **Answer:** It is stored in `.env` on the backend, loaded via `python-dotenv`, and never exposed to the React frontend or pushed to git repositories.

J. Error Handling**Q17: How does your backend handle malformed JSON from the AI model?**

- **Answer:** `groq_service.py` wraps `json.loads()` in a `try...except JSONDecodeError` block and raises an HTTP 400 with a descriptive error message.

K. Architecture**Q18: What is Separation of Concerns in your backend?**

- **Answer:** Isolating routing (**routes/**), business logic (**services/**), data schemas (**schemas/**), and configuration (**config.py**) into separate modules for maintainability and clean code architecture.

14. "WHY" QUESTIONS (QUICK REVISION GUIDE)

1. WHY FASTAPI?

Answer:* High performance, native async support, automatic Pydantic validation, and instant Swagger docs.

One-Liner:* Fast, async-native Python framework with automatic data validation and API documentation.

2. WHY PYDANTIC?

Answer:* Enforces type safety at runtime, eliminating manual payload validation logic.

One-Liner:* Runtime type validation and automatic schema enforcement.

3. WHY REACT?

Answer:* Component-based declarative UI architecture with efficient reactive DOM updates.

One-Liner:* Component-based dynamic UI rendering without page reloads.

4. WHY GROQ?

Answer:* Delivers ultra-low latency LLM inference using custom LPU hardware.

One-Liner:* Ultra-fast AI inference engine for instant recipe generation.

5. WHY QWEN MODEL (qwen/qwen3.8-27b)?

Answer:* High-reasoning open-weights model supporting strict JSON structured mode on Groq Cloud.

One-Liner:* Reliable, high-accuracy chat model supporting strict JSON mode.

6. WHY POST METHOD FOR RECIPES?

Answer:* Form inputs are passed securely inside the JSON request body rather than URL parameters.

One-Liner:* Safely transmits payload inside request body without URL size limits.

7. WHY APIRouter?

Answer:* Keeps **main.py** clean by modularizing endpoints into separate domain files.

One-Liner:* Modularizes routes for clean code architecture.

8. WHY SEPARATION OF CONCERNS?

Answer:* Ensures route handlers, AI logic, data models, and configs can be updated independently.

One-Liner:* Decouples routes, services, and schemas for maintainability.

9. WHY ENVIRONMENT VARIABLES?

Answer:* Keeps sensitive credentials out of source control.

One-Liner:* Prevents secret API keys from leaking to public source code.

10. WHY STRUCTURED JSON RESPONSE?

Answer:* Allows frontend components to reliably bind and render specific UI fields (ingredients list, steps).

One-Liner:* Guarantees predictable data shapes for UI rendering.

11. WHY CORSMiddleware?

Answer:* Permits cross-origin HTTP calls from port 5173 to port 5000.

One-Liner:* Enables browser-allowed requests between frontend and backend origins.

12. WHY SWAGGER UI (/docs)?

Answer:* Provides interactive, self-documenting API testing directly in the browser.

One-Liner:* Interactive browser UI for testing backend endpoints.

15. REALISTIC PROJECT CHALLENGES & RESOLUTIONS

Q1: What was the biggest challenge during backend migration?

Answer: Migrating from Express.js to FastAPI required restructuring the backend architecture to leverage Python's async/await model and Pydantic validation schemas. Furthermore, establishing a custom exception handler was critical to ensure FastAPI error responses maintained the exact **{"error": "..."} JSON** payload structure expected by the existing React frontend.

Q2: What issue did you encounter with the Groq AI model, and how was it solved?

Answer: Originally, the project referenced **llama-3.1-8b-instant**, which returned a **404 model_not_found** error due to model decommissioning on Groq Cloud. I queried the active models via the Groq SDK and updated **groq_service.py** to use **qwen/qwen3.8-27b**. I verified that **qwen/qwen3.8-27b** fully supported **response_format={"type": "json_object"}** and successfully validated it with automated test requests.

16. TESTING STRATEGY

Endpoints Verified via Test Suite

1. **GET /:** Verified **200 OK** status and welcome JSON message.
 2. **GET /health:** Verified **200 OK** status and `{"status": "ok"}` body.
 3. **GET /docs:** Verified OpenAPI Swagger UI loads.
 4. **POST /api/recipe/dish:** Tested with valid payload `{"dishName": "Pasta Carbonara"}` $\rightarrow$ Verified 200 OK and structured JSON recipe response.
 5. **POST /api/recipe/ingredients:** Tested with `{"ingredients": "Eggs, Cheese, Pasta, Guanciale"}` $\rightarrow$ Verified 200 OK.
 6. **Validation Failure:** Tested empty inputs $\rightarrow$ Verified **400 Bad Request** with `{"error": "..."}` payload.
 7. **CORS Verification:** Preflight **OPTIONS** request checked for **Access-Control-Allow-Origin: http://localhost:5173**.
-

17. DEPLOYMENT BASICS

- **Frontend Deployment:** Host static React build on Vercel or Netlify; configure **VITE_API_BASE_URL** env variable to production backend URL.
 - **Backend Deployment:** Deploy FastAPI container/application on Render, Railway, or AWS EC2 using Uvicorn. Set **GROQ_API_KEY** and **PORT** inside cloud platform settings.
 - **Production CORS:** Update **origins** list in **main.py** to allow the production domain (e.g., `https://mohits-kitchen.vercel.app`).
-

18. RESUME EXPLANATION & PITCHES

Resume Bullet Points

- Built **Mohit's Kitchen**, a full-stack AI recipe generation web app using **React (Vite)**, **Python FastAPI**, and **Groq Cloud API (qwen/qwen3.8-27b)**.
- Migrated legacy backend from Node.js/Express to **FastAPI & Pydantic**, implementing async route handling, strict type validation, and custom error middleware.
- Engineered structured JSON prompt workflows with **Groq LPU acceleration**, reducing inference latency and guaranteeing 100% predictable recipe UI rendering.

30-Second Elevator Pitch

"Mohit's Kitchen is a full-stack AI recipe generator built with React and Python FastAPI. It allows users to enter a dish name or available pantry ingredients to receive custom, structured step-by-step recipes. The backend uses Pydantic for validation and communicates asynchronously with Groq Cloud API using the Qwen 27B model to return strict, ready-to-render JSON data."

"Tell Me About Your Project" Interview Answer

"I built Mohit's Kitchen to solve the everyday problem of food waste and pantry indecision. The frontend is built with React, Vite, and Framer Motion for a fluid user interface. For the backend, I migrated from Express to Python FastAPI for enhanced performance and native async handling. The backend validates requests using Pydantic schemas and passes them to a dedicated Groq Service. We connect to Groq Cloud API running the Qwen 27B model with JSON mode enabled, ensuring the AI returns clean, structured recipe data containing instructions, ingredients, cooking time, and difficulty level."

19. RAPID REVISION NOTES

- **FastAPI** $\rightarrow$ High-performance, async Python web framework.
- **Pydantic** $\rightarrow$ Runtime data validation and typing library.
- **Uvicorn** $\rightarrow$ Asynchronous ASGI web server.
- **APIRouter** $\rightarrow$ Router utility for modularizing endpoint routes.
- **Groq Cloud** $\rightarrow$ LPU hardware platform for low-latency LLM inference.
- **Qwen 27B** $\rightarrow$ Chat model (**qwen/qwen3.8-27b**) configured for recipe generation.
- **JSON Mode** $\rightarrow$ `response_format={"type": "json_object"}` forcing raw JSON responses.
- **CORS Middleware** $\rightarrow$ Allows browser requests from **localhost:5173** to **localhost:5000**.
- **Swagger UI** $\rightarrow$ Auto-generated interactive API docs at **/docs**.
- **.env** $\rightarrow$ Environment variable file storing hidden **GROQ_API_KEY**.

20. FINAL INTERVIEW CHEAT SHEET

```

MOHIT'S KITCHEN INTERVIEW CHEAT SHEET
TECH STACK: React 18, Vite, FastAPI, Pydantic, Uvicorn, Groq (Qwen 27B)
KEY ENDPOINTS: POST /api/recipe/dish | POST /api/recipe/ingredients
HEALTH CHECK: GET /health | SWAGGER DOCS: GET /docs
TOP 5 KEY ANSWERS TO REMEMBER:
1. Why FastAPI? Async native, auto Pydantic validation, instant /docs.
2. Why Pydantic? Runtime type checking & automatic request validation.
3. Why Groq? Ultra-fast LPU inference speeds for real-time AI responses.
4. Why JSON Mode? Guarantees parsable raw JSON string output.
5. How is Security Handled? GROQ_API_KEY stored in .env via python-dotenv

```